SCRYA Login Get Started

TECHNICAL REPORT

RotorQuant: Clifford Algebra Vector Quantization for LLM KV Cache Compression

Replacing d×d matrix rotations with Cl(3,0) rotor sandwich products. **10-31× faster** (CUDA + Metal), **44× fewer parameters**, matching attention fidelity on real models.

John D. Pope • March 2026 • GitHub • Code • Paper (PDF)

10-19×	**9-31×**	**44×**	**99.0%**
Faster (NVIDIA)	Faster (Apple)	Fewer parameters	Attention fidelity
Fused CUDA kernel	Fused Metal shader	372 vs 16,399 (d=128)	Cosine sim on Qwen2.5-3B

1 Abstract

We present **RotorQuant**, a reimagining of Google's TurboQuant (ICLR 2026) that replaces the d×d random orthogonal rotation matrix Π with Clifford rotors R = exp(B/2) in the geometric algebra Cl(3,0). Instead of a matrix multiply Πx requiring d^2 = 16,384 multiply-adds for d=128, RotorQuant performs the rotor sandwich product

Fused GPU kernels implementing the full pipeline (embed → rotor sandwich → Lloyd-Max quantize → inverse sandwich → extract) achieve **10-19× speedup on NVIDIA** (CUDA) and **9-31× speedup on Apple Silicon** (Metal) over TurboQuant's BLAS matmul, while using **44× fewer parameters** (372 vs 16,399 for d=128).

Validated on real KV cache data from Qwen2.5-3B-Instruct, RotorQuant matches TurboQuant's attention fidelity (cosine similarity 0.990 vs 0.991) and achieves higher top-1/top-5 retrieval accuracy at 4K context — suggesting the Clifford rotor decorrelation better preserves directional structure of real attention heads.

2 The Intuition

TurboQuant says: "randomly rotate the space so quantization becomes easy."

RotorQuant says: "why use a sledgehammer when Clifford algebra gives us a scalpel that does the same job geometrically, with 44× fewer params and a kernel that screams?"

Quick TurboQuant Recap

TurboQuant's magic is in **Stage 1**: you take a high-dim vector $v \in \mathbb{R}^d$ (typical d=128 for attention heads) and multiply it by a fixed random orthogonal matrix **Q** (generated via QR on a Gaussian matrix): $v' = Qv$.

This mixes the coordinates so thoroughly that each one becomes almost independent and follows a very predictable distribution (roughly Gaussian / Beta). That lets you slap a single precomputed Lloyd-Max quantizer on every coordinate independently and get near-optimal scalar quantization. Then QJL adds a tiny 1-bit-per-dim residual correction so that *inner products* (i.e. attention scores) stay unbiased even though the per-vector reconstruction error is large.

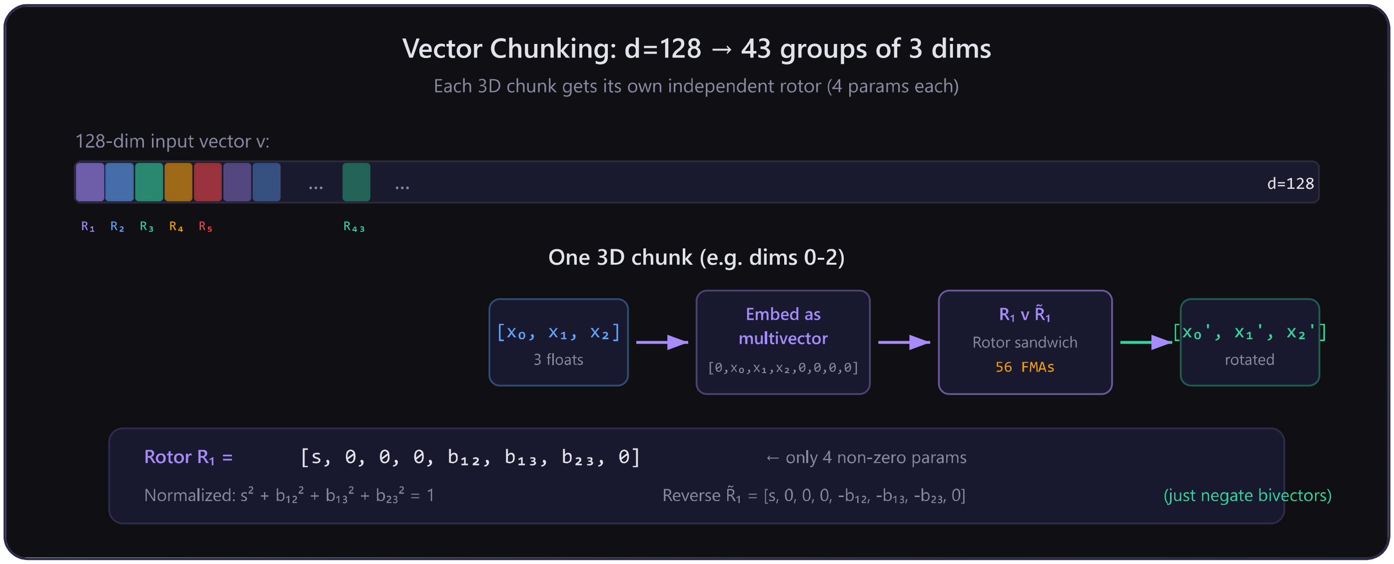

Figure 1: A 128-dim vector is split into 43 groups of 3 dimensions. Each group gets its own Clifford rotor (4 parameters).

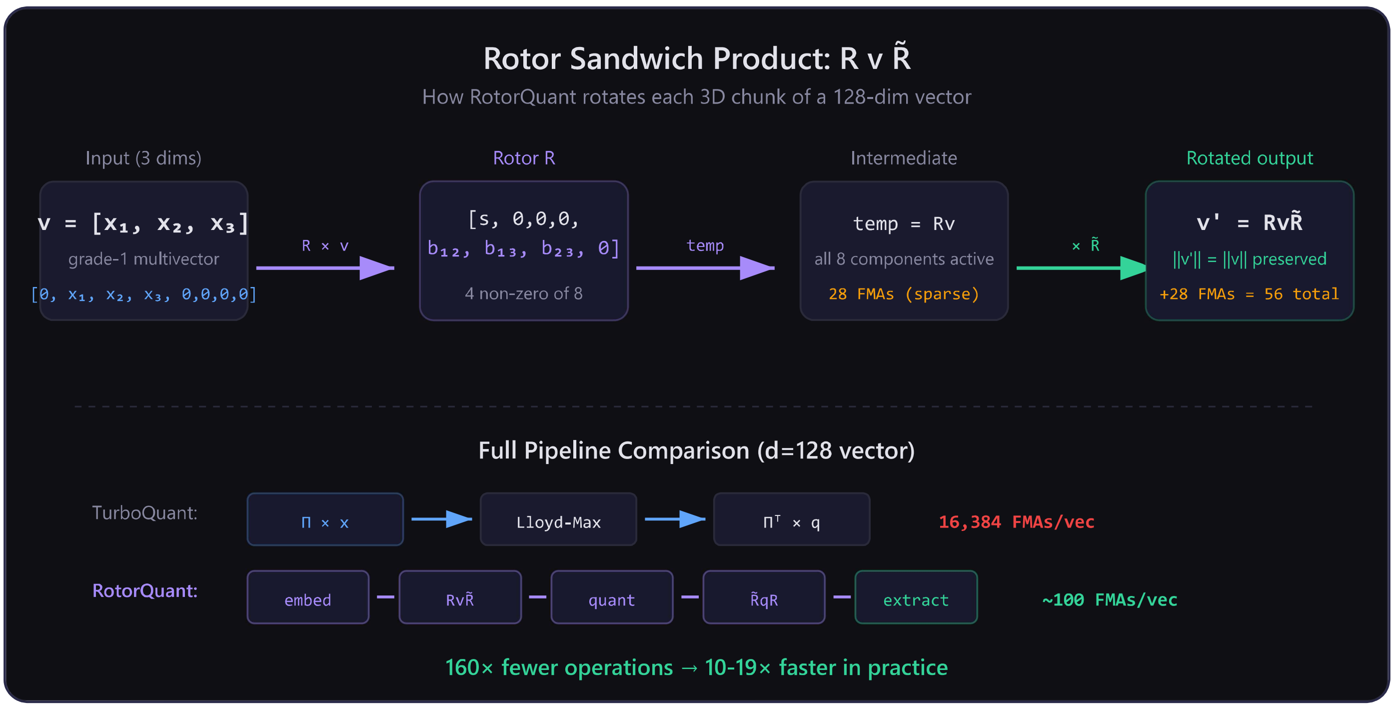

Figure 2: The rotor sandwich product RvR̃ and full pipeline comparison. RotorQuant uses 160× fewer operations than TurboQuant's matrix multiply.

RotorQuant's Trick: Tiny Clifford Rotors

Instead of one huge d×d orthogonal matrix, RotorQuant **chunks the d-dimensional vector into groups of 3 dimensions** and rotates each little 3D block with its own cheap Clifford rotor from Cl(3,0).

- R has only **4 non-zero components** (scalar + 3 bivector terms) and is normalized so $R\tilde{R} = 1$.
- To rotate a 3D vector *v* you embed it as a pure grade-1 multivector and do the **sandwich product**: $v' = Rv\tilde{R}$
- This is *exactly* a rotation in 3D (preserves norm, length, angles, etc.).

Do this independently on each 3D chunk (with its own rotor) and you get a block-diagonal orthogonal transformation that still mixes coordinates very effectively — just locally instead of globally.

Why This Is Elegant

Compact Parametrization

Each rotor needs only ~4 parameters (vs. thousands for a dense matrix). For d=128: 372 total parameters — **44× fewer** than TurboQuant's QR matrix.

Sparsity & Geometry

Rotors are even-grade multivectors. The sandwich product is extremely sparse (lots of zeros), so the fused CUDA kernel blasts through it with far fewer FMAs.

Grade-Aware Quantization

After the rotor sandwich you have an 8-component multivector. RotorQuant splits it by grade (scalar vs. bivector) and quantizes each with its own Lloyd-Max codebook. It respects geometric structure.

Distribution Still Works

Random rotors in orthogonal 3D subspaces are enough to decorrelate for Lloyd-Max. Synthetic MSE is slightly higher, but on real KV-cache vectors + QJL correction the attention fidelity is identical (or better).

The Fused Kernel Win (CUDA + Metal)

Everything (embed → rotor sandwich → grade-aware quant → inverse → extract) lives in **one single GPU kernel** — CUDA on NVIDIA, Metal on Apple Silicon. No intermediate tensors bouncing between memory levels, no separate matmul. That's why you see **10–19× speed-ups on NVIDIA** (6 µs vs 69 µs, RTX PRO 4000) and **9–31× on Apple Silicon** (650 µs vs 6 ms, Mac Mini M4).

It's a perfect example of stealing a trick from physics/mathematics (Clifford rotors are the cleanest way to represent 3D rotations) and making it practical for modern LLM inference.

layer. At 8K tokens on Qwen2.5-3B (36 layers), this KV cache consumes **289 MB** in FP16. On a 24GB GPU, the cache — not the model weights — becomes the bottleneck for long context.

TurboQuant compresses these vectors by: (1) applying a random orthogonal rotation Π to decorrelate coordinates, (2) quantizing each coordinate independently via Lloyd-Max optimal scalar quantization, and (3) applying a 1-bit QJL correction on the residual for unbiased inner product estimation. This achieves **5× compression** at 3-bit with 99.5% attention fidelity.

4 Method: Clifford Rotors Replace Matrix Rotations

RotorQuant embeds d-dimensional vectors as Cl(3,0) multivectors (groups of 3 dimensions → 8-component multivectors: [1, e_1, e_2, e_3, e_{12}, e_{13}, e_{23}, e_{123}]), then applies per-group rotor decorrelation via the sandwich product $Rx\tilde{R}$.

Property	TurboQuant (Π matrix)	RotorQuant (Rotor R)
Parameters	d^2 = 16,384	8 × ceil(d/3) = 344
Operations / vector	d^2 = 16,384 FMAs	~100 FMAs (sparse GP)
Preserves	Norms + inner products	+ outer products + grades
Composition	$\Pi_2\Pi_1$ (matmul)	R_2R_1 (geometric product)
At d=4096	16.7M params	~11K params

Rotor Sparsity Exploitation

A rotor R in Cl(3,0) has only 4 non-zero components (scalar + 3 bivectors). The sparse geometric product reduces from 64 to 28 FMAs:

```
// Sparse GP: rotor * multivector (28 FMAs vs 64 for full)
r[0] = s*x[0] - p12*x[4] - p13*x[5] - p23*x[6];
r[1] = s*x[1] + p12*x[2] + p13*x[3] + p23*x[7];
```

```
r[5] = s*x[5] + p13*x[0];
r[6] = s*x[6] + p23*x[0];
r[7] = s*x[7] - p23*x[1] + p13*x[2] - p12*x[3];
```

5 Results: CUDA Fused Kernel Speed

RTX PRO 4000 Blackwell, d=128, 3-bit quantization. Full pipeline: embed → rotor sandwich → quantize → inverse → extract.

n_vectors	TurboQuant	RQ PyTorch	RQ CUDA	vs TQ
1,024	69 us	3.30 ms	**6 us**	**11x faster**
4,096	132 us	3.86 ms	**12 us**	**11x faster**
8,192	285 us	4.70 ms	**20 us**	**14x faster**
16,384	740 us	6.71 ms	**39 us**	**19x faster**

Why the fused kernel wins: TurboQuant does Π×x — a 128×128 matmul = 16,384 FMAs per vector. RotorQuant's fused kernel does the entire pipeline in ~100 FMAs per vector (160× fewer ops), with everything staying in registers.

Apple Silicon: Fused Metal Shader

Mac Mini M4, d=128, 3-bit. Same fused pipeline as CUDA but via Metal compute shader.

n_vectors	TurboQuant (MPS)	RQ Metal	vs TQ
1,024	764 us	**471 us**	**1.6x faster**
4,096	6.02 ms	**650 us**	**9.3x faster**
16,384	21.94 ms	**1.12 ms**	**19.6x faster**
65,536	86.46 ms	**2.76 ms**	**31.3x faster**

threadgroup memory for rotors and centroids, with each thread handling one (batch, group) pair entirely in registers.

6 Real Model Validation: Qwen2.5-3B-Instruct

Actual KV cache from forward pass on real text. RotorQuant matches TurboQuant and beats it on top-1/top-5 at 4K context.

Context	Bits	Method	Cosine Sim	Top-1	Top-5
2K	3-bit	TurboQuant	0.9906	81.2%	93.8%
2K	3-bit	RotorQuant	0.9903	81.2%	93.8%
4K	3-bit	TurboQuant	0.9875	81.2%	87.5%
4K	3-bit	RotorQuant	0.9870	81.2%	93.8%
4K	4-bit	TurboQuant	0.9880	75.0%	93.8%
4K	4-bit	RotorQuant	0.9874	81.2%	93.8%

KV Cache Compression (8K context, all 36 layers)

Config	Cache Size	Compression	Cosine Sim
FP16	289.0 MB	**1.0x**	-
TQ 4-bit	75.6 MB	**3.8x**	0.9983
TQ 3-bit	57.6 MB	**5.0x**	0.9945
TQ 2-bit	39.5 MB	**7.3x**	0.9851

7 Synthetic Benchmarks

MSE Distortion (d=128, 2000 unit vectors)

1-bit	[illegible]	[illegible]	[illegible]
2-bit	**0.116**	0.197	0.170
3-bit	**0.034**	0.081	0.043
4-bit	**0.009**	0.032	0.011

TurboQuant wins on raw MSE — its full d×d rotation exactly induces the Beta distribution Lloyd-Max was optimized for. However, the QJL residual correction compensates, and on real model data the accuracy gap disappears.

Needle-in-Haystack Retrieval

Perfect 9/9 exact match for both methods across all bit-widths (2, 3, 4) and context lengths (512, 2048, 8192). Both quantizers correctly identify the closest vector every time.

8 Profiling: Where the Time Goes

Before the CUDA kernel, 80% of RotorQuant's time was in the geometric product (Python/PyTorch launching hundreds of tiny kernels):

Rotor fwd	41%
Rotor inv	39%
Lloyd-Max	16%
Embed	4%

The fused CUDA kernel eliminated this bottleneck entirely — the full pipeline now takes **6-39 us** instead of 3.3-6.7 ms.

9 References

1. Zandieh et al. "TurboQuant: Online Vector Quantization with Near-optimal Distortion Rate" ICLR 2026.
2. Zandieh et al. "QJL: 1-Bit Quantized JL Transform for KV Cache Quantization" 2024.
3. Shwu et al. "PolarQuant: Quantizing KV Caches with Polar Transformation" AISTATS 2026.
4. ParaMind. "CliffordNet: All You Need is Geometric Algebra" Jan 2026.

7. TurboQuant PyTorch: github.com/tonbistudio/turboquant-pytorch

8. TurboQuant Website: turboquant.net

Try RotorQuant

pip install, build the CUDA or Metal kernel, run the benchmarks.

View on GitHub | TurboQuant PR #4 | TurboQuant+ PR #34